The Claude Prompting Guide

What this guide combines:

1. The **Code w/ Claude 2026** talks: San Francisco (May 6), London (May 19–20) and Tokyo (June 10–11).
2. Anthropic's official **Prompting best practices** documentation.
3. Anthropic's engineering articles on **context engineering**, **writing tools for agents**, **Agent Skills** and **commerce agents**.

It's organized by **topic, not by talk**. When several talks made the same point, the point appears once, with every source credited.

Source key

Tag	Source
📘	Anthropic docs: *Prompting best practices*
🧩	Anthropic engineering: *Effective context engineering*, *Writing tools for agents*, *Agent Skills*
🛒	Anthropic: *The anatomy of effective commerce agents*
🎤 Keynote	Opening keynote (Ami Vora, Dianne Penn, Angela Jiang, Katelyn Lesse, Cat Wu, Boris Cherny)
🎤 Curve	*The capability curve* (Alex Albert)
🎤 Thinking	*The thinking lever*
🎤 Toolkit	*The expanding toolkit* (Lucas Gonzalez)
🎤 Platform	*Getting more out of the Claude Platform* (Brad Abrams)
🎤 GitHub	*Caching, harnesses, and advisors* (Mario Rodriguez, GitHub; Brad Abrams)
🎤 Playbook	*The prompting playbook* (Margot van Laar in London; Hannah Moran in SF and Tokyo)
🎤 Models	*Picking the right model*
🎤 Memory	*Memory and dreaming for self-learning agents* (Mahesh Murag)
🎤 CMA	*How to get to production faster with Claude Managed Agents* (Jess Yan, Lance Martin)
🎤 Bun	*Live coding session* (Boris Cherny, Jarred Sumner)
🎤 Vercel	*Architecting for model step-changes* (Guillermo Rauch)
🎤 Panel	*Building AI-native: Cognition, Gamma, Harvey*
🎤 Cursor	*Giving coding agents their own computers*
🎤 Replit	*Evaluating and improving Replit Agent at scale*
🎤 Decompose	*Tool, skill, or subagent? Decomposing an agent that outgrew its prompt* (Will Steuk, Tanveer Mittal)
🎤 Basics	*Beyond the basics with Claude Code* (Daisy Hollman)

> **How the talk content was gathered:** for the 19 SF recordings I used a public set of machine-made transcripts (github.com/PiLastDigit/Code-With-Claude). For London and Tokyo talks with no public transcript, I used the official session descriptions plus third-party recaps. Those points are marked *(recap)*. The talks demo Opus 4.7, Sonnet and Haiku, which were current in May 2026. The principles carry over to newer models, but check the model-specific docs page for your model.

1. Mindset

1.1 Treat Claude like a brilliant new hire 📘🧩

- It's smart, but it knows nothing about your norms, goals or workflow, so spell them out.
- **Colleague test:** if a coworker with no background would be confused by your prompt, Claude will be too.
- Onboard an agent the way you'd onboard a developer: give it a computer, docs, a way to run the app, and test accounts. 🎤 Cursor

1.2 Prompts can't add capabilities 🎤 Playbook *(recap)*

- Shouting "CALCULATE CORRECTLY" at a bot that miscalculates doesn't fix it. **Give it a tool** (code execution or a calculator) instead.
- Use evals to tell apart **behavior problems**, which a prompt can fix, from **capability gaps**, which no prompt fixes.

1.3 Design for the *next* model, not the current one 🎤 Keynote 🎤 Curve 🎤 Panel

- Make model upgrades **cheap**: automated evals, simple scaffolding, and ambitious prototypes that don't quite work yet. When one of them suddenly starts passing, that's your signal.
- Expect to re-architect. Harvey rebuilt at each of three capability jumps. Cognition **deleted** its custom code-editing and planning systems once models could edit, grep and plan natively.

1.4 Give the model room to work 🎤 Curve 🎤 Vercel

- New models **plan before acting**, **recover from dead ends** without looping, and **stay coherent** over hundreds of thousands of tokens. So:
  - Don't force them to jump straight into action. Let them think.
  - Don't chunk work artificially or babysit the context window.
  - **Don't micromanage.** Vercel found fewer fixed tools and subagents plus a sandbox worked better. "You don't want to be the manager that's like, hey, did you check this?"
- Grant **more tool access, in a controlled way**. For example, Claude Code's auto mode uses a classifier to decide which calls need a human.

1.5 Close the loop 🎤 Curve 🎤 Bun 🎤 Keynote

- Design the system so Claude can **check its own output**: tests, a browser, computer use, screenshots, logs.
- "If Claude can check its work, you can just let it run." 🎤 Keynote

2. Writing the prompt

2.1 Clear, specific, direct 📘

- Say exactly what output you want, in what format, within what limits.
- When order or completeness matters, use **numbered steps**.
- If you want ambition, **ask for it**. Claude won't infer it.
  - ❌ `Create an analytics dashboard`
  - ✅ `…Include as many relevant features and interactions as possible. Go beyond the basics to create a fully-featured implementation.`

2.2 Explain *why* 📘🎤 Playbook

- A rule with a reason generalizes. A bare rule doesn't.
  - ❌ `NEVER use ellipses` → ✅ `Your response will be read aloud by a text-to-speech engine, so never use ellipses since it can't pronounce them.`
- Give **both sides of a trade-off**. In one support-bot prompt, escalation was described only as "costs money and counts against the team," never mentioning its upside, so the bot under-escalated. *(recap)*

2.3 The right "altitude" 🧩

- Aim between two failures: **brittle if-else rules** and **vague guidance** that assumes context Claude doesn't have.
- **Minimal doesn't mean short.** Include everything needed, and nothing that isn't.
- **Start minimal on the best model**, then add instructions and examples only to fix failures you actually see in testing.

2.4 Structure 📘🧩🎤 Playbook

- Keep **role, rules, data and task** in clearly separated sections. *(recap)*
- Use XML tags (`<instructions>`, `<context>`, `<input>`, `<documents>`) or Markdown headers. Keep tag names consistent and nest them when there's a hierarchy.

2.5 Role 📘

- One sentence in the system prompt focuses tone and behavior: `You are a helpful coding assistant specializing in Python.`

2.6 Examples (few-shot) 📘🧩

- The most reliable way to steer format, tone and structure.
- Make them **relevant**, **diverse** (include edge cases) and **wrapped** in `<example>` / `<examples>` tags. Aim for **3–5**.
- Prefer a few **canonical examples** over a long list of edge-case rules.
- Examples shape thinking too. Show **problem → method → answer**.
- You can ask Claude to critique your examples or write more of them.

2.7 Long documents (20k+ tokens) 📘

- **Data at the top, question at the end.** In tests this improved quality by up to 30%.
- Wrap each document: `<document index="n"><source>…</source><document_content>…</document_content></document>`.
- **Ground the answer in quotes:** have Claude extract relevant quotes into `<quotes>` first, then answer from them.

2.8 Say what *to* do 📘

- ❌ "Don't use markdown" → ✅ "Write in smoothly flowing prose paragraphs."
- **Match your prompt's style** to the output you want. A prompt full of markdown gets markdown back.

2.9 Calm language 📘

- New models follow the system prompt closely. `CRITICAL: You MUST…` now causes **overuse**. Write `Use this tool when…` instead.

2.10 Model identity (for apps) 📘

- `The assistant is Claude, created by Anthropic. The current model is …` If your app picks a model, give it the exact model string.

3. Controlling the output

3.1 Formatting 📘

- Use XML output tags (`<smoothly_flowing_prose_paragraphs>`), or give explicit formatting rules: prose over bullets, lists only for truly separate items, headings and code blocks only.
- Plain math: `Do not use LaTeX… use "/" for division, "*" for multiplication, "^" for exponents.`

3.2 Length and progress updates 📘

- Recent models are concise and may skip summaries after tool use. If you want one, ask: `After completing a task that involves tool use, provide a quick summary.`
- Models differ (Opus 5 runs long; Fable 5.1 writes fewer progress updates). Prompt for the length you want. Don't rely on the effort setting to change it.

3.3 Prefill is gone; use these instead 📘

You used to prefill to…	Do this now
Force JSON/YAML	**Structured outputs**, or ask for the schema plus retries. For classification, use a tool with an `enum`.
Skip "Here is…"	`Respond directly without preamble.`, XML tags, or tools. Strip any leftover preamble in code.
Avoid refusals	A clear user message is enough now.
Continue cut-off text	In the user turn: `Your previous response was interrupted and ended with […]. Continue from where you left off.`
Re-inject context	Put it in the user turn, hydrate it through tools, or use compaction.

3.4 Act vs. suggest 📘

- `Can you suggest changes…` → Claude only suggests. `Change this function…` → Claude makes the change.
- System-prompt switches: `<default_to_action>` (implement by default; use tools to find missing details instead of guessing) or `<do_not_act_before_instructions>` (research and recommend unless told to edit).

4. Thinking and effort 🎤 Thinking 📘

4.1 How it works

- More **test-time compute** means more intelligence, the same way more training compute does.
- Claude spends three kinds of tokens: **thinking** (its scratchpad), **tool calls** (acting on the world) and **text** (talking to you).
- It went from *think-then-act*, to *interleaved thinking* between tool calls, to **adaptive thinking**, where it thinks, acts and writes in whatever order works best.
- **Effort dials beat thinking on/off switches.** Effort governs the whole workflow (thinking, tool use and output), not just one part. Use **task budgets** to cap tokens, time or cost on long tasks.
- `budget_tokens` is deprecated (it errors on 4.7 and later). Use adaptive thinking plus `effort`, and `max_tokens` as a hard cap.

4.2 Picking an effort level

Level	Use for
Max	Your hardest tasks. Test it, but expect diminishing returns. One level down is often nearly as good for much less.
Extra high	**Default for coding and agentic work.** It's the default in Claude Code and claude.ai. If you have no evals and you're doing software work, pick this.
High	Balanced. A good starting point for intelligence-sensitive work; test upward from here.
Medium	Cost-sensitive work where you'll trade a little intelligence for speed.
Low	Short, latency-sensitive tasks. It **takes shortcuts** to save tokens, sometimes cleverly (on low effort, Claude Plays Pokémon "speed-ran" the game), sometimes not. Read the transcripts.

4.3 Big model on low effort, or small model on high?

- **Small models:** fastest *time to first token*. Best for bulk, simple work like classification, extraction and basic summaries.
- **Big model at low effort:** fastest *time to last token* on tasks that need intelligence. Opus at low effort beat Haiku at max effort on roughly the same tokens.
- A smarter model from the start beat a smaller model with maximum thinking. 🎤 Models *(recap)*

4.4 Steering thinking with prompts 📘

- **General beats prescriptive.** "Think thoroughly" often beats a hand-written step-by-step plan.
- Reflect after tools: `After receiving tool results, carefully reflect on their quality and determine optimal next steps before proceeding.`
- Less thinking: `Thinking adds latency… When in doubt, respond directly.` Or lower the effort.
- Stop flip-flopping: `Choose an approach and commit to it… you can course-correct later.`
- Self-check: `Before you finish, verify your answer against [criteria].` (Remove this on models that already over-verify, such as Opus 5.)

4.5 The three actions 🎤 Thinking

1. **Turn thinking on.** Control it with effort or budgets.
2. **Use evals:** chart performance against tokens, time or cost across effort levels and models, and **read the transcripts**.
3. **No evals and doing coding?** Pick extra high.

5. Tools

5.1 Pick and design tools 🧩🛒

- **More tools isn't better.** Don't just wrap every API endpoint. If a human couldn't tell two tools apart, Claude can't either.
- **Targeted beats dump-everything:** `search_contacts` over `list_contacts`.
- **Merge chained steps** into one tool (`schedule_event`, `get_customer_context`), and return only the relevant log lines.
- **Namespace** related tools (`asana_projects_search`). Test whether a prefix or suffix works better for your model.
- **Wrap your real systems.** Don't reimplement their logic in the tool. When domain logic piles up, move it to one backend endpoint. 🛒

5.2 What tools should return 🧩🛒🎤 Toolkit

- **Return high-signal fields**: names and types, not UUIDs or MIME types. Turning cryptic IDs into readable ones **reduced hallucinations**.
- Add a `response_format` parameter (`concise` / `detailed`).
- Paginate, filter and truncate with sensible defaults. When truncating, **tell Claude how to narrow its search**. (Claude Code caps tool output at 25k tokens.)
- **Errors should be actionable**, with an example of correct input, not a bare code or traceback.
- **Describe the output schema** in the tool definition so Claude can anticipate results and skip extra round-trips. 🎤 Toolkit
- Drop noise such as image URLs. Add a "next step" when it isn't obvious. 🛒

5.3 Describe tools like you'd onboard a new hire 🧩

- Explain formats, jargon and how resources relate to each other. Use clear parameter names (`user_id`, not `user`) and strict schemas.
- Small description tweaks can produce big gains; refining descriptions alone helped reach state-of-the-art on SWE-bench.
- Use MCP annotations to flag destructive or open-world tools.
- **Build tools with Claude:** prototype, run evals, paste the transcripts into Claude Code, and let it refactor the tools. Hold out a test set.

5.4 Tool behavior 📘

- **Parallel calls:** Claude already runs independent calls in parallel. A `<use_parallel_tool_calls>` block pushes this to nearly 100% (dependent calls stay sequential; never guess parameters). To slow down: `Execute operations sequentially…`
- **Over-triggering:** remove `If in doubt, use [tool]`. Write `Use [tool] when it would enhance your understanding.`

5.5 Platform tools that save context 🎤 Platform

- **Tool search:** declare hundreds of tools but load them **just in time**. Lovable cut tokens by about 10%, and the model got **smarter** with less clutter.
- **Programmatic tool calling:** Claude writes Python that calls tools, keeps the results in memory and prints only what it needs.
- **Code execution sandbox:** hosted, so you don't run the write-run-fix loop yourself. 🎤 Toolkit

5.6 CLIs, sandboxes and "a computer for every agent" 🎤 Vercel 🎤 Basics 🎤 Cursor

- With a sandbox, tools can **emerge**: Claude writes its own helper scripts instead of you pre-building dozens of tools.
- If a **CLI already does the job**, let Claude shell out to it. Many MCP servers with many tools crowd out the actual task. *(recap)*
- Teach unfamiliar CLIs with a **skill**.
- Give agents **eyes**: a browser, screenshots, logs.

6. Context engineering 🧩🎤 Platform 🎤 Toolkit

6.1 Principles

- Context is a **finite attention budget**. Every token costs some attention.
- **Context rot:** recall degrades gradually as context grows.
- The goal is **the smallest set of high-signal tokens**.
- **Look at the raw transcript.** Frameworks that hide what Claude actually sees make it impossible to optimize. 🎤 Platform

6.2 Load just in time

- Give Claude **references** (file paths, queries, links) and let it fetch what it needs with `grep`, `head` and `tail`, instead of pasting everything.
- **Hybrid:** load the essentials up front (like `CLAUDE.md`) and explore the rest on demand. "Do the simplest thing that works."
- **File systems are replacing RAG and custom planning systems.** "Write your plan down and follow it" now just works. 🎤 Panel

6.3 Keep context clean on long runs

Technique	What it does
Clear stale tool results	Drop old screenshots and search dumps but keep the decisions they led to. 🎤 Toolkit
Compaction	Summarize stale turns (the "sledgehammer"). Keep decisions, open bugs and key details. Tune for recall first, then precision.
Lower compaction threshold	You don't need the full 1M tokens. A lower threshold can cut cost and latency and even **raise** intelligence. 🎤 Platform
Structured notes	`NOTES.md` or to-do files outside the context, reloaded later. Claude Plays Pokémon kept maps and tallies across thousands of steps.
Subagents	Explore in a clean context and return a 1–2k-token summary.

6.4 Long context isn't automatically pricier 🎤 GitHub

- Small windows force **more compactions**. Each compaction produces about 4k output tokens (which cost about 5× input) and breaks the cache. Manage compaction deliberately.

7. Architecture: prompt, tool, skill, subagent, or second model?

7.1 Where each piece of logic belongs 🛒🧩🎤 Decompose

Put it in…	When
System prompt	Critical everywhere (safety, legal, brand, key user facts like allergies), or relevant to roughly **⅓ or more of requests**.
Tool	A deterministic action or lookup. Don't write prompt text for something a function can compute.
Skill	Domain know-how needed only sometimes. It loads into the **same** agent, which keeps the full history.
Subagent	A narrow, self-contained job that needs its own big context and returns a short answer (deep research, parallel work, independent review).
Handoff	Only when another agent should **own the conversation** (for example, its own compliance rules).

- In Anthropic's enterprise comparisons, **one agent with skills** beat both one giant prompt and one-agent-per-domain, often on cost and latency too. 🛒
- Subagent handoffs **lose state**, cost **several times more tokens** and add **seconds of latency**. 🛒
- Over-spawning: `Use subagents when tasks can run in parallel, require isolated context, or involve independent workstreams… For simple tasks… work directly.` 📘
- **Decomposition in practice:** an inherited 402-line agent was broken apart live, with evals re-run after every change. Common root causes were **missing tools**, **mismatched output formats** between pieces, and **contradictory rules** in the prompt. 🎤 Decompose *(recap)*
- **Split a failing mega-prompt** into simple stages, such as **draft → check → repair**. 🎤 Playbook *(recap)*

7.2 How to build a skill 🧩🛒

- A skill is a folder containing `SKILL.md`, with YAML front matter (`name`, `description`) and optional scripts and resources.
- **Progressive disclosure:** only the name and description sit in context; the body loads when relevant; extra files load only when opened.
- **The description is the trigger.** Write it carefully.
- Bundle **scripts** for deterministic work (sorting, parsing). The code runs without its source entering context. Say whether a script is meant to be *run* or *read*.
- Build skills **from evals**: find where the agent struggles, add a skill, and repeat. Have Claude record what worked and what went wrong.
- Load skills as **tool results**, not by editing the system prompt, so the cache survives. If you can predict the skill from a known signal, inject it before the first call. 🛒
- **Security:** install only from trusted sources, and audit scripts and any instructions to reach the network.

7.3 Pairing models 🎤 Keynote 🎤 GitHub 🎤 Platform

- **Advisor strategy:** a small model (Haiku or Sonnet) does the work and calls **Opus as a tool** only on hard steps, giving near-Opus quality at a fraction of the cost.
- **Critique ("rubber duck"):** insert a reviewer model at three points: **after the plan, after complex implementation, and before running tests**. 🎤 GitHub
- **Chaining** 📘: separate calls for draft → review against criteria → refine, when you need to inspect the intermediate steps.

7.4 Managed Agents primitives 🎤 Keynote 🎤 CMA

- **Outcomes:** you write a **rubric**, and a separate grader in its own context sends work back until it passes. Up to about 10 points of lift on Anthropic's hardest internal problems.
- **Multi-agent orchestration:** a lead agent delegates to specialist subagents, each with its own model, prompt and tools, on a shared filesystem.
- **Inner loop / outer loop:** Outcomes improve each run (the inner loop). You plus a coding agent refine the instructions and rubric between runs (the outer loop). One demo cut a render from 37s to 10s.

8. Caching and cost 🎤 Platform 🎤 GitHub 🛒

- Caching gives a **90% discount** on input tokens, faster time to first token, and **cached tokens don't count against rate limits**.
- **Target 90%+ hit rates.** GitHub runs above 94%; Cursor, Replit and Perplexity are in the 90s.
- **Order by stability:** system prompt → tools → session history → volatile data **at the very end**.
- **No dynamic content in the prefix.** A UUID in GitHub's system prompt silently broke its whole cache. Swapping tools in and out also breaks it, so add regression tests.
- Keep history **append-only**. Editing past turns or rebuilding `system` / `tools` invalidates the cache and thinking blocks. 📘
- **Tools:** the Console's prompt-cache dashboard, and the built-in Claude Code skill: tell it *"improve my cache hit rate."*
- **Measure outcomes, not activity:** code *survival rate* over *acceptance rate*, and **cost per completed task**, not cost per call. 🎤 GitHub 🛒

9. Evals: how you know a prompt works 🎤 Curve 🎤 Replit 🎤 Models 🛒🧩

9.1 Build the right evals

- **Measure your real traffic**, not academic benchmarks. Treat public benchmarks as industry averages and your evals as your own KPIs.
- **Keep them unsaturated.** Make them harder as models improve.
- **Grade the end state, not the path:** set up state, add the user message, run, then grade the final state and output, including the arguments of the last write. 🛒
- Every positive case gets a **negative twin** (act vs. ask, serve vs. refuse). Include **messy, contradictory** histories.
- Start with **50–100 cases per user flow**, built with experts and from real incidents.
- **Good tool evals** need many tool calls, have a verifiable outcome, and don't fail correct answers over formatting. 🧩
- **"Taste" can be evaluated:** use rubric-driven, replayable evals built from real projects that track quality, cost, latency, errors and tokens. 🎤 *Evals for taste (recap)*

9.2 Run them continuously

- **Change nothing until you can measure it.** 🎤 Playbook *(recap)*
- **Offline:** benchmarks before release. **Online:** cluster production traces to find failure patterns, then A/B test. When metrics conflict, human product judgment decides. 🎤 Replit
- **CI:** core cases, all safety cases, and cases touching the change. The **full suite nightly.** Gate on pass rate over several trials. Roll out to a small group first, with a kill switch per skill. 🛒
- **Read transcripts.** Scores don't show shortcuts or odd behavior. 🎤 Thinking 🧩

9.3 Hill-climbing 🎤 Bun

- Give Claude **a metric, a way to verify and a target**, then let it iterate in auto mode until it gets there.

10. Picking models and upgrading 🎤 Models 🎤 Curve 🎤 Thinking 📘

- The right model is the **cheapest per successful outcome** on *your* evals, not the cheapest per token and not automatically the smartest. *(recap)*
- **Sweep model × thinking × effort** across your evals and plot the curves.
- Often the **best optimization is swapping in the newest model**, but upgrades aren't drop-in. Plan for evals, prompt rework, A/B tests and sometimes orchestration changes.
- **On every upgrade, prune:**
  - **Scaffolding:** a multi-step workflow may now be one thread. Removing things often **raises** performance.
  - **Prompt cruft:** rules pile up "model over model." Cut the ones you can't explain. An old defensive rule made a newer model **withhold information it had**. 🎤 Playbook *(recap)*
  - **Anti-laziness text:** "be thorough" and "always use tools" now cause overuse. 📘
- **Migration checklist** 📘: describe the exact behavior you want; add quality modifiers; request features like animation explicitly; switch to adaptive thinking + effort; remove prefill; pass thinking blocks back unchanged; read your model's specific docs page.

11. Claude Code and agentic coding

11.1 CLAUDE.md is "compound engineering" 🎤 Bun 🎤 Basics

- Put in **build commands, how to write and run tests, where tests go, and past pitfalls**.
- **Rule: anything you've repeated once or twice goes into** `CLAUDE.md`, so Claude gets it right the first time from then on.
- Keep it short, limited to what's costly to forget. *(recap)*
- Claude Code's building blocks: `CLAUDE.md`, MCP, skills, hooks, subagents, plugins. 🎤 Basics *(recap)*

11.2 Verification first 🎤 Bun 🎤 Cursor

- **No PR without a passing test that reproduces the bug.** Bun's bot reproduces every issue before a human looks at it.
- Layer reviewers: a style checker for `CLAUDE.md` rules plus Claude code review for subtle, whole-codebase bugs.
- Have agents **report friction** (confusing setup, broken tools) into a log, then fix the "factory." 🎤 Cursor

11.3 Workflow features 🎤 What's new 🎤 Keynote

- **Plan mode** first on big changes. **Worktrees** for parallel tasks. **Auto mode** instead of constant permission prompts.
- **Auto memory** (`MEMORY.md`) keeps preferences and build commands across sessions.
- **Routines are "a higher-order prompt."** Instead of prompting Claude yourself, you set up a routine (cron, webhook or API trigger) that prompts it. Example: a routine that babysits a PR, fixing review comments and CI and rebasing.

11.4 Prompt snippets for coding agents 📘

Problem	Add this
Risky actions	Ask before destructive, hard-to-reverse or public actions (`rm -rf`, `push --force`, PR comments). Never bypass checks (`--no-verify`).
Over-engineering	`Only make changes that are directly requested… no extra features, docstrings, defensive code, or one-off abstractions.`
Gaming tests	`Implement a general solution… don't hard-code test values… tell me if tests are wrong.`
Hallucinated code	`<investigate_before_answering>`: never speculate about code you haven't opened.
Leftover scratch files	`Clean up temporary files at the end of the task.`
Generic UI	Ask for distinctive typography and color, motion and atmosphere. Avoid Inter or Roboto and purple-on-white gradients.
Vision tasks	Give Claude a **crop/zoom** tool.
Research	Give success criteria, ask it to verify across sources, and have it track competing hypotheses with confidence in a notes file.

12. Long-running agents and memory

12.1 Work that spans several context windows 📘

1. Use a **different first prompt** that sets up the framework: tests, `init.sh`.
2. Track tests in **`tests.json`**. Add: *"It is unacceptable to remove or edit tests."*
3. A **fresh window often beats compaction:** start with `pwd`, read `progress.txt`, `tests.json` and `git log`, and run one integration test.
4. Tell Claude about compaction: `…do not stop tasks early due to token budget concerns… save progress to memory before the window refreshes.`
5. Ask it to **use the whole budget**, without leaving significant uncommitted work.
6. State: **JSON** for structured data, **free text** for notes, **git** for checkpoints. Push for **incremental progress**.

12.2 Memory design 🎤 Memory 🛒

- **Memory works as a file system:** Claude organizes it with `bash` and `grep`. Use read-only vs. read-write scopes, version history and audit logs.
- **Dreaming:** an offline process that reviews past sessions, finds recurring mistakes and team preferences, and curates memory without slowing live runs. Rakuten saw **90% fewer first-pass mistakes**; Harvey saw **6× task completion** on one scenario.
- Keep a **small always-on set** in context, pre-fetch facts from known signals, and put everything else behind a lookup tool.
- Write memory **asynchronously**, with an extractor that reads only user and assistant text, **never tool results** (that's an injection risk). 🛒

12.3 Safety boundaries 🛒🎤 Panel

- The model **proposes** changes; your system **approves and applies** them. Accept only server-issued IDs, and **clean third-party text** before it reaches the model.
- Keep hard data boundaries between clients and projects (Harvey's "ethical walls").

13. One-page cheat sheet

1. **Brief a new hire:** clear, specific, with the *why*, in the right altitude.
2. **Separate role, rules, data and task** with XML tags. Long data first, question last.
3. **3–5 diverse examples.** Say what *to* do. Calm wording, no CRITICAL/MUST.
4. **Capability comes from tools, not pleading.** Give Claude code execution, a browser, tests.
5. **Turn thinking on and control it with effort.** For coding, use extra high by default.
6. **Few, well-named tools** that return lean, readable output and actionable errors.
7. **Engineer context:** load just in time, clear stale results, compact deliberately, keep notes.
8. **Always-relevant → prompt · deterministic → tool · sometimes-needed know-how → skill · isolated or parallel job → subagent · hard steps → advisor model.**
9. **Cache:** stable prefix, volatile data last, append-only history, 90%+ hits.
10. **Evals on real traffic.** Grade outcomes, add negative twins, read transcripts, measure cost per success.
11. **Every new model: re-test, then delete** scaffolding and prompt cruft.
12. **Close the loop.** Give Claude a metric and a way to verify, then let it run.

Appendix: sessions and sources

Recorded SF talks used (transcripts): Opening keynote · What's new in Claude Code · Live coding (Boris Cherny & Jarred Sumner) · Caching, harnesses, and advisors (GitHub) · Managed Agents to production · Building AI-native (Cognition, Gamma, Harvey) · Getting more out of the Claude Platform · Datadog machine tools · The capability curve · Vercel fireside · Asana AI teammates · Running an AI-native engineering org · The thinking lever · Claude on Google Cloud · Replit evals · Cursor cloud agents · Memory and dreaming · The expanding toolkit · Dario & Daniela Amodei.

London and Tokyo sessions used via official descriptions and recaps: The prompting playbook · Picking the right model · Beyond the basics with Claude Code · Stop babysitting your agents · Agents that remember · Evals for taste · Tool, skill, or subagent?

On the agenda but with no public content I could find: How we Claude Code · Ship your first Managed Agent · Agent Battle · Personality goes a long way (thinking like a writer) · How Metaview built self-improving prompts · Making agentic workflows trustworthy with a custom DSL (Elicit) · Designing multi-agent systems (Mintlify) · Evals for subjective, stateful agents (Descript) · Build a proactive agent workflow · Designing with Claude.

Links

- Agendas: claude.com/code-with-claude/san-francisco, …/london, …/tokyo (plus `-extended` pages)
- Transcripts: github.com/PiLastDigit/Code-With-Claude (third-party, machine-made)
- Prompting best practices: platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices
- anthropic.com/engineering/effective-context-engineering-for-ai-agents
- anthropic.com/engineering/writing-tools-for-agents
- anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
- claude.com/resources/articles/the-anatomy-of-effective-commerce-agents